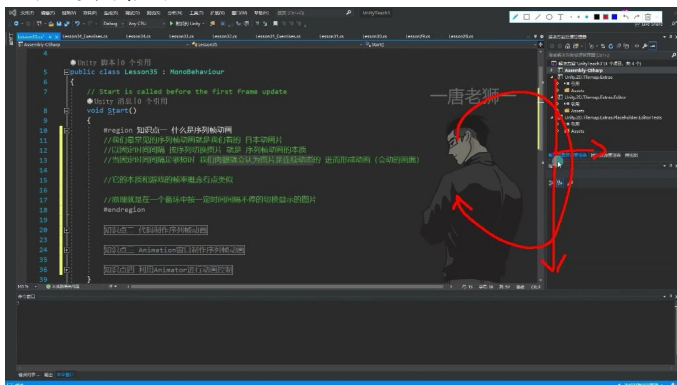


一、序列帧动画 00:18

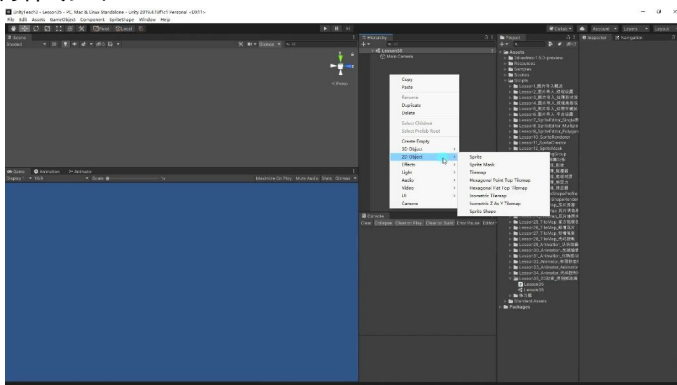
1. 什么是序列帧动画 00:41



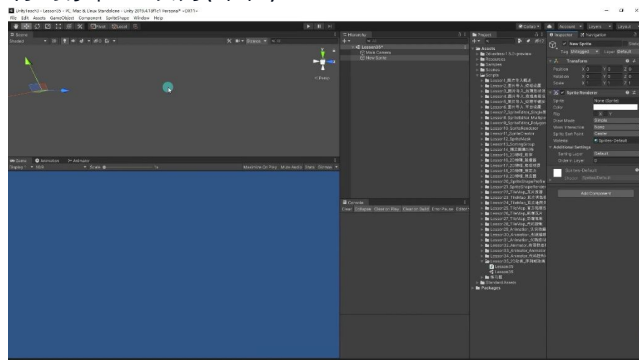
- 原理：通过快速切换连续图片产生动态效果，类似翻书动画
- 实现方式：以固定时间间隔按序列切换图片，当间隔足够短（如30毫秒）时，人眼会认为是连续动画
- 类比：类似游戏帧率概念，Unity中通过循环不断更新画面
- 经典示例：日本动画片就是典型的序列帧动画应用

2. 代码制作序列帧动画 01:57

1) 制作动画 02:11

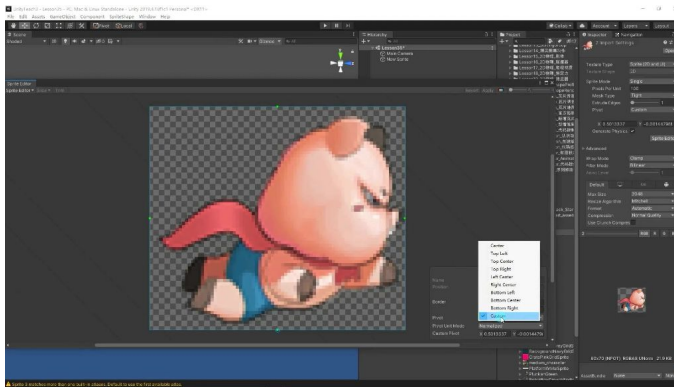


- 准备工作:
- 新建空场景
- 创建2D Sprite对象
- 将对象位置设为(0,0,0)

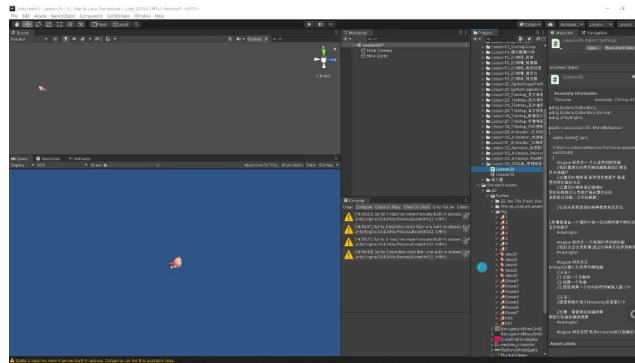


-
- **核心思路:**
 - 定时修改Sprite Renderer上的图片信息
 - 通过代码控制图片切换频率

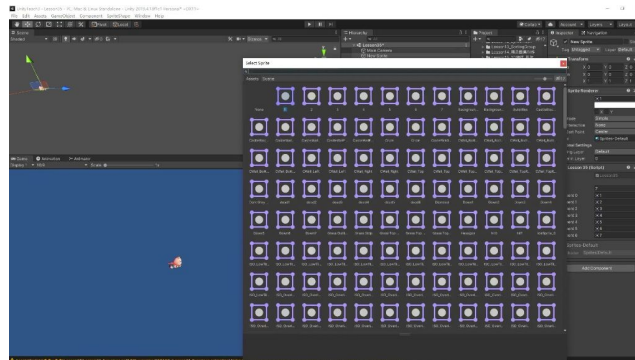
- 使用Sprite数组存储动画帧序列
- ## 2) 编辑图片轴心点 02:53



- 关键设置：
 - 确保所有图片轴心点居中
 - 避免因轴心点不一致导致位置偏移
 - 在Sprite Editor中统一设置并应用



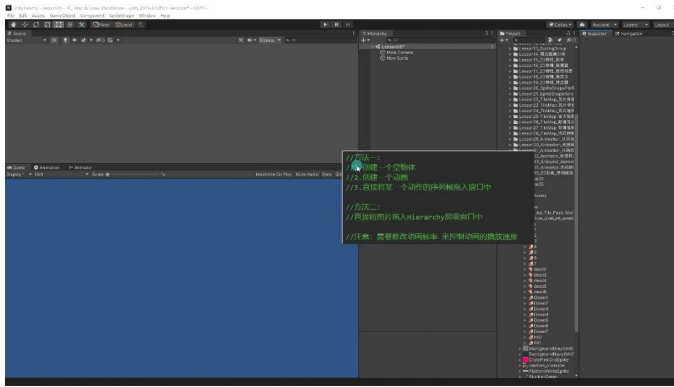
- 代码实现：
- 参数说明：
 - 0.03f表示30毫秒切换一帧
 - nowIndex记录当前显示帧索引
 - 数组越界时重置为0实现循环播放



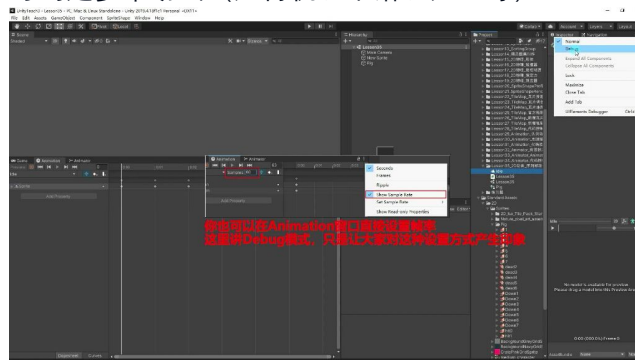
- 测试要点：
 - 将脚本挂载到Sprite对象
 - 在Inspector面板关联7张序列图片
 - 运行后观察连续动画效果
 - 可通过调整时间间隔控制动画速度

3. Animation窗口制作序列帧动画 08:21

1) 方法一 08:35

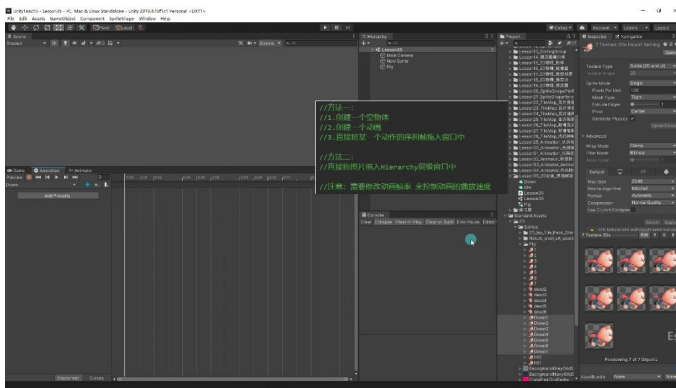


- 步骤:
 - 创建一个空物体 (如命名为"Pig")
 - 打开Animation窗口创建新动画 (如"待机"动画)
 - 将动作序列帧全选后拖入Animation窗口
- 帧率调整技巧:
 - 先删除已有关键帧
 - 在Inspector窗口切换Debug模式
 - 修改Sample Rate值 (如从60改为15)
 - 重新拖入序列帧图片
- 动画编辑:
 - 可手动拖动关键帧调整顺序
 - 可拉伸关键帧间距改变播放速度
 - 建议使用帧率统一控制速度 (避免逐帧调整)
- 注意事项:
 - 修改帧率前务必先删除关键帧
 - 动画文件会自动关联Animator组件
 - 可创建多个动画 (如待机、下落、死亡等)

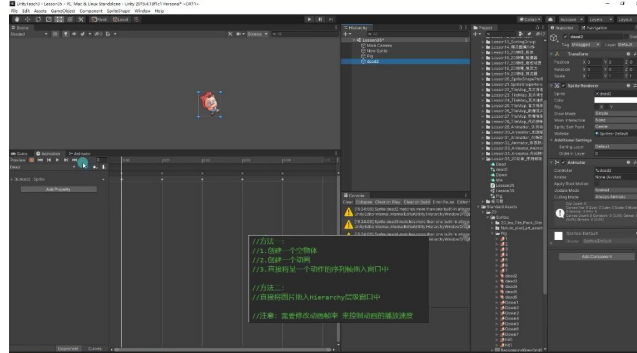


- 帧率设置方法:
 - 选中动画文件
 - 切换Inspector为Debug模式
 - 修改Sample Rate参数
 - 典型值: 15帧/秒 (较慢) 或60帧/秒 (默认)

2) 方法二 13:36



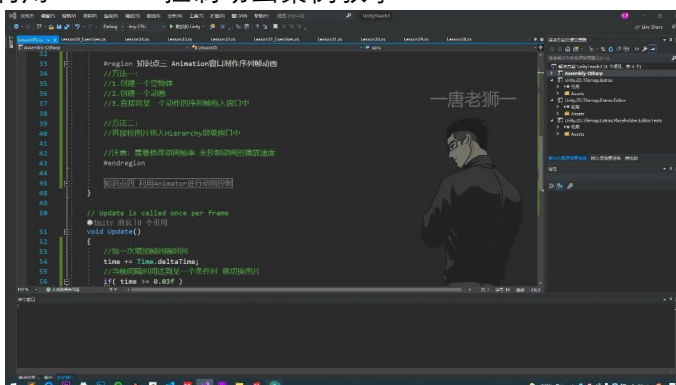
- 操作流程:
 - 直接拖拽序列帧图片到Hierarchy窗口
 - 弹出保存对话框时命名动画 (如"Dead")
 - 系统自动创建GameObject和Animator
- 特点:
 - 适用于创建第一个动画
 - 自动生成包含SpriteRenderer的对象
 - 后续动画仍需使用方法一创建
- 限制:
 - 不适用于添加后续动画
 - 生成的对象名称固定为第一张图片名
 - 需要手动调整初始位置



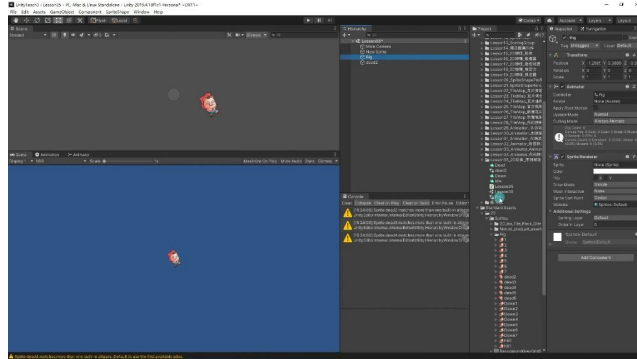
- 验证方法:
 - 选中生成的对象
 - 在Animation窗口查看自动创建的动画
 - 可播放预览效果
 - 仍需通过Debug模式调整帧率

4. 利用Animator进行动画控制 15:12

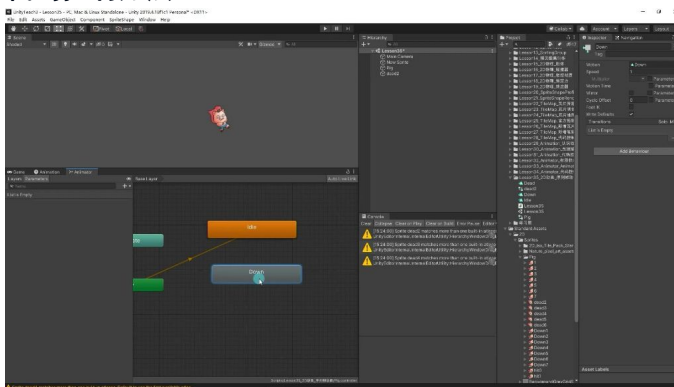
1) 利用Animator控制动画案例教学 15:22



- **状态切换原理：**通过Animator控制器建立不同动画状态之间的转换关系，使用条件参数控制状态切换
- **条件设置方法：**
 - 在状态转换线上添加条件参数（如布尔值）
 - 示例中设置"isDown"参数控制待机和下落状态的切换
 - true时切换到下落状态，false时返回待机状态

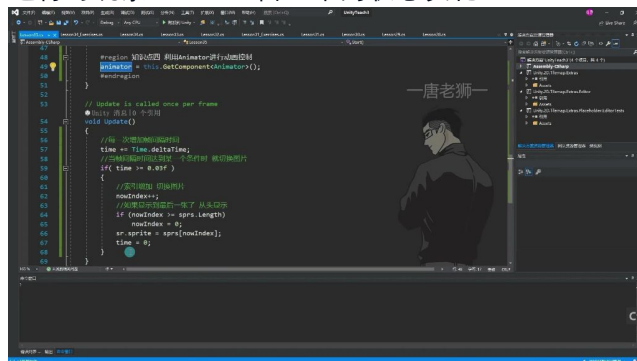


- **重要设置项：**
 - 取消勾选"Has Exit Time"确保条件满足时立即切换
 - 仅无条件自动切换时才保留"Has Exit Time"选项
- **代码控制方法：**



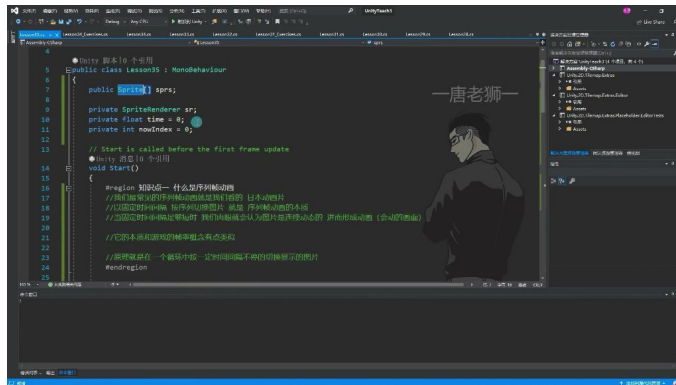
- **实际应用技巧：**
 - 动画控制通常需要配合移动逻辑代码
 - 示例中使用空格键控制状态切换：

- **调试技巧：**
 - 通过Inspector窗口拖拽方式获取Animator组件
 - 运行时观察Animator窗口中的状态变化

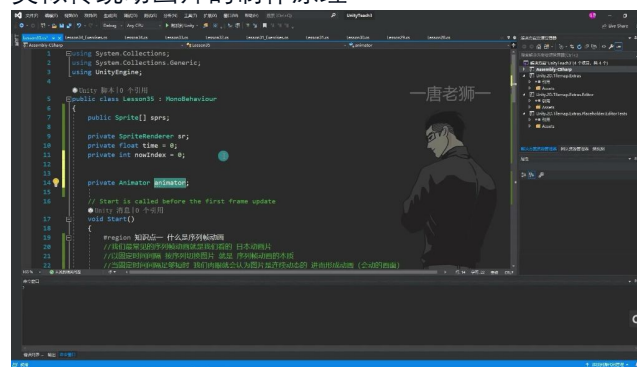


- **注意事项：**
 - 动画帧率需要单独设置控制播放速度
 - 复杂动画需要设计合理的状态转换规则
 - 2D动画通常配合SpriteRenderer组件使用

2) 序列帧动画制作方法



-
- **本质原理：**
 - 以固定时间间隔按序列切换图片
 - 间隔足够短时形成连续动画效果
 - 类似传统动画片的制作原理

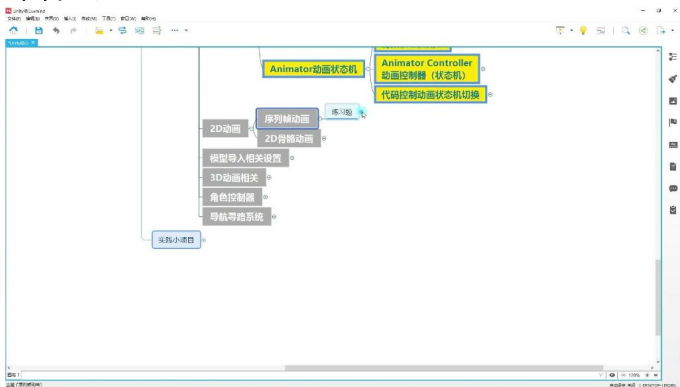


-
- **制作方法一：**
 - 创建空物体
 - 创建Animation剪辑
 - 将序列帧拖入Animation窗口
- **制作方法二：**
 - 直接将图片拖入Hierarchy窗口自动生成动画
- **关键参数：**
 - 必须调整动画帧率控制播放速度

3) 综合应用建议

- **开发实践：**
 - 动画模块应与游戏逻辑分离
 - 典型应用场景：
 - 角色移动状态切换
 - 攻击动作表现
 - 环境特效动画
- **扩展思路：**
 - 结合物理系统实现更真实的动画效果
 - 使用动画事件触发游戏逻辑
 - 混合使用代码动画和Animator控制

二、课后习题 20:36



- 练习内容：在之前2D相关练习题的基础上，为控制的对象加上序列帧动画
- 练习方式：根据课程内容进行截图练习
- 讲解安排：老师将在下一个视频中对该习题进行详细讲解



- 知识要点：
 - 掌握Animator Controller动画控制器的使用
 - 理解动画状态机(Animator)的工作原理
 - 能够通过代码控制动画状态机的切换
 - 熟悉序列帧动画的制作方法
- 练习重点：
 - 将之前学习的2D控制逻辑与动画系统相结合
 - 实现基本的序列帧动画效果
 - 理解2D动画与3D动画在Unity中的区别
- 后续内容预告：
 - 2D骨骼动画的实现
 - 模型导入相关设置
 - 3D动画系统
 - 角色控制器与导航寻路系统
 - 实践小项目开发

三、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
序列帧动画原理	通过连续切换图片形成动画效果，类比翻页书或日本动画片。本质是 固定时间间隔切换图片 ，与游戏帧率循环原理相似。	关键理解时间间隔与动画流畅度的关系（如30	★★

		毫秒切换)。	
代码实现序列帧动画	1. 声明Sprite数组存储帧图片; 2. 定时修改SpriteRenderer的sprite属性; 3. 通过索引循环切换图片 (需处理越界重置)。	注意 轴心点对齐 问题, 否则动画位置会偏移。	★★★
Animation窗口制作序列帧动画	方法一: 空物体+创建动画+拖入序列帧; 方法二: 直接拖图片到层级窗口 (仅首次动画适用)。; 关键技巧 : 通过修改动画文件帧率 (如15fps) 控制播放速度。	必须先删 关键帧 再改帧率, 避免帧丢失问题。	★★
Animator动画控制	1. 状态机中设置布尔条件 (如isDown); 2. 代码调用SetBool切换状态; 3. 取消Has Exit Time确保即时响应。	条件触发与动画过渡的优先级关系 (无条件时才用Exit Time)。	★★★
综合应用	动画模块需结合移动逻辑 (如按键控制), 表现与逻辑分离 。示例: 空格键切换飞行/下落动画。	多动画管理时注意状态机复杂度 (连线规则按需求设计)。	★★★★